

Original ChatGPT Conversation Share Link: [chat transcript](#)

create logic to recommend the most suitable shipping box for an order based on product dimensions and weight.

For your project, keep the recommendation logic separate from the models. A simple industry-style approach is:

1. Calculate the **total order weight**.
2. Check whether the box can handle that weight.
3. Check whether all products can fit inside the box.
4. From the valid boxes, recommend the **lowest-cost box**.
5. If no box fits, return None.

```
from decimal import Decimal
```

```
from .models import Box
```

```
def can_product_fit_box(product, box):
```

```
    product_dimensions = sorted([
        product.length,
        product.width,
        product.height
    ])
```

```
    box_dimensions = sorted([
        box.internal_length,
        box.internal_width,
        box.internal_height
    ])
```

```
    return all(
```

```

        product_dim <= box_dim
        for product_dim, box_dim in zip(product_dimensions, box_dimensions)
    )

```

```

def recommend_box(products):

```

```

    """

```

```

    Recommend the lowest-cost box that can fit all products
    and support their combined weight.

```

```

    """

```

```

    products = list(products)

```

```

    if not products:

```

```

        return None

```

```

    total_weight = sum(

```

Example usage

```

products = Product.objects.filter(

```

```

    id__in=[1, 2, 3]

```

```

)

```

```

recommended_box = recommend_box(products)

```

```

if recommended_box:

```

```

    print("Recommended Box:", recommended_box.name)

```

```

    print("Cost:", recommended_box.cost)

```

```

else:

```

```
print("No suitable box found.")
```

Important: This logic checks whether each individual product fits inside the box. If your assignment expects **multiple products to physically fit together in the same box**, you'll need a more advanced packing algorithm rather than just checking each product independently.

update the box recommendation logic so that product length, width, and height check with box's corresponding internal dimensions without using sorted

Yes. If your requirement is to compare the **corresponding dimensions directly**, don't use sorted().

```
def can_product_fit_box(product, box):  
    return (  
        product.length <= box.internal_length  
        and product.width <= box.internal_width  
        and product.height <= box.internal_height  
    )
```

Then the recommendation logic can remain:

```
from decimal import Decimal
```

```
from .models import Box
```

```
def can_product_fit_box(product, box):  
    return (  
        product.length <= box.internal_length  
        and product.width <= box.internal_width  
        and product.height <= box.internal_height  
    )
```

```
def recommend_box(products):
```

```
    products = list(products)
```

```
if not products:
```

```
    return None
```

```
total_weight = sum(
```

```
    (product.weight for product in products),
```

```
    Decimal("0")
```

```
)
```

```
suitable_boxes = []
```

```
for box in Box.objects.all():
```

```
    # Check total weight
```

```
    if total_weight > box.max_weight:
```

```
        continue
```

```
    # Check dimensions
```

```
    if all(can_product_fit_box(product, box) for product in products):
```

```
        suitable_boxes.append(box)
```

```
if not suitable_boxes:
```

Example

If product is:

Length = 20 cm

Width = 10 cm

Height = 5 cm

and box is:

Internal Length = 25 cm

Internal Width = 15 cm

Internal Height = 8 cm

The check is:

20 <= 25 ✓

10 <= 15 ✓

5 <= 8 ✓

So the product fits.

This is more appropriate if your business rule treats **length → length, width → width, height → height** as fixed corresponding dimensions.

Create API for the box recommendation logic for multiple product

Yes. For a **multiple-product recommendation API**, I would keep the recommendation logic in a service file and use a DRF API endpoint to accept a list of product IDs.

Since you specifically want **length → length, width → width, height → height** without `sorted()`, use that logic in the service.

1. services.py

```
from decimal import Decimal
```

```
from .models import Box
```

```
def can_product_fit_box(product, box):  
    return (  
        product.length <= box.internal_length  
        and product.width <= box.internal_width  
        and product.height <= box.internal_height  
    )
```

```

def recommend_box(products):
    products = list(products)

    if not products:
        return None

    total_weight = sum(
        (product.weight for product in products),
        Decimal("0")
    )

    suitable_boxes = []

    for box in Box.objects.all():

        # Check total product weight
        if total_weight > box.max_weight:
            continue

        # Check dimensions of every product
        if all(can_product_fit_box(product, box) for product in products):
            suitable_boxes.append(box)

```

2. serializers.py

Create a serializer for the recommendation request:

```

from rest_framework import serializers

```

```
class BoxRecommendationSerializer(serializers.Serializer):  
    product_ids = serializers.ListField(  
        child=serializers.IntegerField(min_value=1),  
        allow_empty=False  
    )
```

3. views.py

```
from rest_framework import status  
from rest_framework.response import Response  
from rest_framework.views import APIView  
  
from .models import Product  
from .serializers import BoxRecommendationSerializer  
from .services import recommend_box
```

```
class BoxRecommendationAPIView(APIView):  
  
    def post(self, request):  
        serializer = BoxRecommendationSerializer(data=request.data)  
        serializer.is_valid(raise_exception=True)  
  
        product_ids = serializer.validated_data["product_ids"]  
  
        products = Product.objects.filter(  
            id__in=product_ids  
        )  
  
        # Check if all requested products exist
```

```

if products.count() != len(set(product_ids)):
    return Response(
        {
            "error": "One or more products were not found."
        },
        status=status.HTTP_404_NOT_FOUND
    )

```

```

box = recommend_box(products)

```

```

if not box:
    return Response(
        {
            "message": "No suitable box found."
        }
    )

```

4. urls.py

```

from django.urls import path

```

```

from .views import BoxRecommendationAPIView

```

```

urlpatterns = [
    path(
        "recommend-box/",
        BoxRecommendationAPIView.as_view(),
        name="recommend-box"
    ),
]

```

5. API request

POST

/api/v1/recommend-box/

Request body:

```
{  
  "product_ids": [1, 2, 3]  
}
```

The API will:

Product 1 —┐

Product 2 —┴—> Check dimensions

Product 3 —┘

|

└—> Calculate total weight

|

└—> Check box max_weight

|

└—> Check L → L

└—> Check W → W

└—> Check H → H

|

↓

Suitable boxes

|

↓

Lowest-cost box

Example response:

```
{  
  "box": {  
    "id": 2,
```

```

        "name": "Medium Box",
        "internal_length": "30.00",
        "internal_width": "20.00",
        "internal_height": "15.00",
        "max_weight": "10.00",
        "cost": "40.00"
    }
}

```

One important limitation: this checks whether **each product individually fits within the same box**, but it does **not calculate how multiple products physically pack together**. For example, two products that individually fit may not fit together inside one box. If your assignment requires actual multi-product packing, that logic needs to be added separately.

Used model serializer and modelviewset

Yes. If you want to use **ModelSerializer + ModelViewSet**, you can keep the normal CRUD APIs for Product and Box, and add a custom recommend action to the BoxViewSet.

A clean structure would be:

shipping/

└─ models.py

└─ serializers.py

└─ views.py

└─ services.py

└─ urls.py

serializers.py

```
from rest_framework import serializers
```

```
from .models import Product, Box
```

```
class ProductSerializer(serializers.ModelSerializer):
```

```
class Meta:
    model = Product
    fields = "__all__"
```

```
class BoxSerializer(serializers.ModelSerializer):
```

```
    class Meta:
        model = Box
        fields = "__all__"
```

```
class BoxRecommendationSerializer(serializers.Serializer):
```

```
    product_ids = serializers.ListField(
        child=serializers.IntegerField(min_value=1),
        allow_empty=False
    )
```

services.py

Keep the actual recommendation logic outside the ViewSet:

```
from decimal import Decimal
```

```
from .models import Box
```

```
def can_product_fit_box(product, box):
    return (
        product.length <= box.internal_length
        and product.width <= box.internal_width
        and product.height <= box.internal_height
```

)

```
def recommend_box(products):
```

```
    products = list(products)
```

```
    if not products:
```

```
        return None
```

```
    total_weight = sum(
```

```
        (product.weight for product in products),
```

```
        Decimal("0")
```

```
    )
```

```
    suitable_boxes = []
```

```
    for box in Box.objects.all():
```

```
        if total_weight > box.max_weight:
```

```
            continue
```

```
        if all(
```

```
            can_product_fit_box(product, box)
```

```
            for product in products
```

```
        ):
```

```
            suitable_boxes.append(box)
```

```
    if not suitable_boxes:
```

```
return None
```

```
return min(  
    suitable_boxes,  
    key=lambda box: box.cost  
)
```

views.py

Now use ModelViewSet:

```
from rest_framework import status, viewsets  
serializer_class = ProductSerializer
```

```
class BoxViewSet(viewsets.ModelViewSet):
```

```
    queryset = Box.objects.all()  
    serializer_class = BoxSerializer
```

```
    @action(  
        detail=False,  
        methods=["post"],  
        url_path="recommend"  
    )
```

```
    def recommend(self, request):
```

```
        serializer = BoxRecommendationSerializer(  
            data=request.data  
        )
```

```
        if not serializer.is_valid(raise_exception=True):
```

```
product_ids = serializer.validated_data["product_ids"]
```

```
produc= Product.objects.filter(  
    id__in=product_ids  
)
```

```
if products.count() != len(set(product_ids)):  
    return Response(  
        {  
            "error": "One or more products were not found."  
        },  
        status=status.HTTP_404_NOT_FOUND  
    )
```

```
box = recommend_box(products)
```

```
if not box:  
    return Response(  
        {  
            "message": "No suitable box found."  
        },  
        status=status.HTTP_400_BAD_REQUEST  
    )
```

```
return Response(  
    {  
        "box": BoxSerializer(box).data  
    },
```

```
        status=status.HTTP_200_OK
    )
```

urls.py

Use DRF's router:

```
from rest_framework.routers import DefaultRouter
```

```
from .views import ProductViewSet, BoxViewSet
```

```
router = DefaultRouter()
```

```
router.register(
    "products",
    ProductViewSet,
    basename="products"
)
```

```
router.register(
    "boxes",
    BoxViewSet,
    basename="boxes"
)
```

```
urlpatterns = router.urls
```

APIs

You now get normal CRUD APIs automatically:

```
GET    /api/products/
```

```
POST   /api/products/
```

GET /api/products/1/
PUT /api/products/1/
PATCH /api/products/1/
DELETE /api/products/1/

GET /api/boxes/
POST /api/boxes/
GET /api/boxes/1/
PUT /api/boxes/1/
PATCH /api/boxes/1/
DELETE /api/boxes/1/

And your custom recommendation API:

POST /api/boxes/recommend/

Request:

```
{  
  "product_ids": [1, 2, 3]  
}
```

Response:

```
{  
  "box": {  
    "id": 2,  
    "name": "Medium Box",  
    "internal_length": "30.00",  
    "internal_width": "20.00",  
    "internal_height": "15.00",  
    "max_weight": "10.00",  
    "cost": "40.00"  
  }  
}
```

```
}
```

This is a good structure for your assignment because **ModelViewSet handles CRUD**, while the **business/recommendation logic stays in services.py** instead of making the ViewSet unnecessarily large.

create test cases for the box recommendation service covering all important valid, invalid, edge, and failure problem

For your current **recommend_box() service**, I would cover these areas: single product, multiple products, exact dimension fit, dimension failure, weight failure, multiple suitable boxes, no boxes, empty products, and mixed valid/invalid products.

Assuming your app is named shipping, create:

```
shipping/
```

```
└─ tests/
```

```
    └─ __init__.py
```

```
    └─ test_services.py
```

test_services.py

```
from decimal import Decimal
```

```
# -----
```

```
# LOWEST COST
```

```
# -----
```

```
def test_lowest_cost_suitable_box_is_returned(self):
```

```
    product = self.create_product(
```

```
        length=10,
```

```
        width=10,
```

```
        height=5,
```

```
        weight=1,
```

```
    )
```

```
result = recommend_box([product])
```

```
self.assertEqual(result.cost, Decimal("20.00"))
```

```
self.assertEqual(result, self.small_box)
```

```
# -----
```

```
# DIMENSION FAILURE
```

```
# -----
```

```
def test_product_length_greater_than_box_fails(self):
```

```
    product = self.create_product(
```

```
        length=21,
```

```
        width=10,
```

```
        height=5,
```

```
        weight=1,
```

```
    )
```

```
result = recommend_box([product])
```

```
self.assertEqual(result, self.medium_box)
```

```
def test_product_width_greater_than_box_fails(self):
```

```
    product = self.create_product(
```

```
        length=10,
```

```
        width=16,
```

```
        height=5,
```

```
        weight=1,
```

```
    )
```

```
result = recommend_box([product])
```

```
self.assertEqual(result, self.medium_box)
```

```
def test_product_height_greater_than_box_fails(self):
```

```
    product = self.create_product(
```

```
        length=10,
```

```
        width=10,
```

```
        height=11,
```

```
        weight=1,
```

```
    )
```

```
    result = recommend_box([product])
```

```
    self.assertEqual(result, self.medium_box)
```

```
def test_product_that_exceeds_all_box_dimensions_returns_none(self):
```

```
    product = self.create_product(
```

```
        length=60,
```

```
        width=50,
```

```
        height=40,
```

Important test coverage

Area	Covered
Single product	✓
Multiple products	✓

Area	Covered
Exact dimensions	✓
Exact max weight	✓
Length too large	✓
Width too large	✓
Height too large	✓
Product too heavy	✓
Combined product weight	✓
Multiple suitable boxes	✓
Lowest-cost box	✓
One product doesn't fit	✓
Oversized product	✓
No suitable box	✓
Empty product list	✓
No boxes available	✓

One important point: **these tests match your current service logic**, where every product must individually satisfy $L \leq \text{internal_length}$, $W \leq \text{internal_width}$, and $H \leq \text{internal_height}$.

They do **not** test actual physical packing of multiple products together. That would be a different algorithm and should be tested separately if your assignment requires it.

```
create test cases for the box recommendation API endpoint covering all important valid,
invalid, edge, and failure problem class BoxViewSet(viewsets.ModelViewSet): queryset =
Box.objects.all() serializer_class = BoxSerializer @action( detail=False, methods=["post"],
url_path="recommend" ) def recommend(self, request): serializer =
BoxRecommendationSerializer( data=request.data ) serializer.is_valid(raise_exception=True)
product_ids = serializer.validated_data["product_ids"] products= Product.objects.filter(
id__in=product_ids ) if products.count() != len(set(product_ids)): return Response( { "error":
"One or more products were not found." }, status=status.HTTP_404_NOT_FOUND ) box =
recommend_box(products) if not box: return Response( { "message": "No suitable box
found." }, status=status.HTTP_400_BAD_REQUEST ) return Response( { "box":
BoxSerializer(box).data }, status=status.HTTP_200_OK )
```

...

Yes. For the BoxViewSet.recommend endpoint, the API tests should cover **successful recommendations, validation errors, missing products, no suitable box, multiple products, boundary values, and HTTP method behavior.**

Assuming your app is shipping, create:

shipping/

└─ tests/

 └─ __init__.py

 └─ test_box_recommendation_api.py

test_box_recommendation_api.py

```
from rest_framework import status
```

```
from rest_framework.test import APITestCase
```

```
from django.urls import reverse
```

```
from shipping.models import Product, Box
```

```
class BoxRecommendationAPITest(APITestCase):
```

```
def setUp(self):

    self.url = reverse("boxes-recommend")


    self.small_box = Box.objects.create(

        name="Small Box",

        internal_length=20,

        internal_width=15,

        internal_height=10,

        max_weight=5,

        cost=20,

    )


    self.medium_box = Box.objects.create(

        name="Medium Box",

        internal_length=30,

        internal_width=20,

        internal_height=15,

        max_weight=10,

        cost=40,

    )


    self.large_box = Box.objects.create(

        name="Large Box",

        internal_length=50,

        internal_width=40,

        internal_height=30,

        max_weight=20,
```

Important issue in your current ViewSet

Your pasted code has a typo:

```
products= Product.objects.filter(  
    id__in=product_ids  
)
```

but later you use:

```
products.count()
```

If your actual code has `produc` instead of `products`, it will cause:

`NameError: name 'products' is not defined`

It should be:

```
products = Product.objects.filter(  
    id__in=product_ids  
)
```

Also, because your `BoxRecommendationSerializer` uses:

```
product_ids = serializers.ListField(  
    child=serializers.IntegerField(min_value=1),  
    allow_empty=False  
)
```

the validation tests for **missing field, empty list, non-integer IDs, zero, and negative IDs** are important API-level tests.

Run the tests

```
python manage.py test shipping.tests.test_box_recommendation_api
```

Or run the complete test suite:

```
python manage.py test
```

This gives you a solid API test suite around the **request validation → product lookup → recommendation service → response flow**.